

SKRENO AUTOMATION · PLAYWRIGHT + TYPESCRIPT

Skreno অটোমেশন সম্পূর্ণ গাইড

ফাঁকা প্রজেক্ট থেকে শুরু করে login test, GitHub Actions CI, login session সেভ করা আর dashboard test পর্যন্ত। প্রতিটা ধাপে কী করবেন আর কেন করবেন, আর পথে যে এররগুলো এসেছিল সেগুলো কীভাবে ঠিক হলো, সব মেন্টরের ভাষায় বাংলায়।

প্রজেক্ট: ~/skreno-automation · Playwright 1.64 · Node 22 · সংস্করণ ২ (৯ অক্টোবর ২০২৬)

সূচিপত্র

1. **পর্ব ১, ভিত্তি:** সেটআপ যাচাই, Git, ফোল্ডার স্ট্রাকচার, `.env`, `config`, `npm scripts`, `Prettier`
2. **পর্ব ২, Login test:** `codegen`, `LoginPage`, `login.spec.ts`
3. **পর্ব ৩, CI:** `GitHub Secrets` আর `workflow`
4. **পর্ব ৪, Session আর Dashboard:** `auth.setup`, `storageState`, `DashboardPage`, `sidebar test`
5. **পর্ব ৫:** কীভাবে নির্দিষ্ট test চালাবেন
6. **পর্ব ৬:** যে এররগুলো এসেছিল আর তার সমাধান
7. **পর্ব ৭:** মেন্টরের পরামর্শ আর পরের ধাপ

চূড়ান্ত প্রজেক্ট স্ট্রাকচার

```
skreno-automation/
├── .github/workflows/playwright.yml → CI
├── pages/
│   ├── LoginPage.ts → login পেজের locator আর action
│   └── DashboardPages.ts → dashboard আর sidebar
├── tests/
│   ├── auth.setup.ts → একবার login করে session সেভ
│   ├── auth/login.spec.ts → login test (logged-out)
│   └── dashboard/navigation.spec.ts → sidebar test (logged-in)
├── utils/dataFactory.ts → faker দিয়ে test data
├── playwright/.auth/user.json → সেভ করা session (git এ যায় না)
├── .env / .env.example → URL, email, password
├── .prettierrc
└── playwright.config.ts
```

ভিত্তি তৈরি

1 সেটআপ ঠিকমতো কাজ করছে কিনা যাচাই

```
cd ~/skreno-automation
npx playwright test --project=chromium
npx playwright show-report
```

কেন: নিজের কোড লেখার আগে নিশ্চিত হয়ে নিন যে টুলটা ঠিকমতো চলছে। example test পাস করলে বুঝবেন Node, Playwright আর ব্রাউজার সব ঠিক আছে। পরে কোনো সমস্যা হলে তখন জানবেন যে দোষটা আপনার কোডে, সেটআপে না।

2 Git চালু করা

```
git init
git add .
git commit -m "Initial Playwright setup"
```

কেন: কিছু ভেঙে গেলে আগের অবস্থায় ফিরে যাওয়ার জন্য git লাগবে। .gitignore আগেই আছে, তাই node_modules আর report ফাইলগুলো commit হবে না।

3 ফোল্ডার স্ট্রাকচার (Page Object Model)

```
mkdir -p pages tests/auth tests/dashboard fixtures utils test-data
git rm tests/example.spec.ts
```

pages/	→ প্রতিটা পেজের locator আর action
tests/	→ আসল test ফাইল (*.spec.ts)
fixtures/	→ custom fixture (পরে লাগবে)
utils/	→ helper function
test-data/	→ স্থির JSON data

কেন: test ফাইলের ভেতরে সরাসরি locator লিখবেন না। Skreno-র UI বদলালে ৫০টা test ফাইল ঠিক করতে হবে, যেখানে POM থাকলে শুধু একটা page ফাইল ঠিক করলেই হয়। এটাকে বলে **Page Object Model**, আর ইন্ডাস্ট্রিতে এটাই স্ট্যান্ডার্ড। example.spec.ts হলো playwright.dev সাইটের test, Skreno-র না, তাই মুছে দিন।

4

.env

দিয়ে গোপন তথ্য আলাদা রাখা

root এ .env ফাইল:

```
BASE_URL=https://www.skreno.ai
TEST_EMAIL=your-test-account@example.com
TEST_PASSWORD=your-password
```

.gitignore এ .env যোগ করুন। আর key গুলোসহ একটা .env.example রাখুন, value ছাড়া।

কেন: পাসওয়ার্ড কখনো কোডে বা git এ রাখবেন না। এছাড়া .env থাকলে staging আর production এর মধ্যে শুধু URL বদলে একই test চালাতে পারবেন।

সতর্কতা: .env এ value '...' কোটেশনের ভেতরে রাখলে dotenv নিজে থেকে কোটেশন বাদ দেয়। কিন্তু GitHub Secrets এ এই value কপি করার সময় কোটেশন কপি করবেন না। বিস্তারিত পর্ব ৩ এ।

5

playwright.config.ts এর মূল সেটিং

ক) dotenv চালু করুন (ফাইলের উপরে)

```
import dotenv from 'dotenv';
import path from 'path';
dotenv.config({ path: path.resolve(__dirname, '.env') });
```

খ) use ব্লক

```
use: {
  baseUrl: process.env.BASE_URL,
  trace: 'retain-on-failure',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
},
```

গ) projects

শুরুতে শুধু একটা সাধারণ chromium project রাখুন। firefox আর webkit কमेंট করে দিন। setup আর storageState যোগ হবে পর্ব ৪ এ।

কেন:

- baseUrl সেট করলে test এ পুরো URL না লিখে শুধু page.goto('/login') লিখলেই হয়।
- test fail করলে screenshot, video আর trace পাবেন। ডিবাগ করার সময় এগুলো সবচেয়ে বেশি কাজে আসে।
- একই নামে দুটো project রাখবেন না। তাহলে প্রতিটা test দুইবার চলবে।

6

package.json scripts

```
"scripts": {  
  "test": "playwright test",  
  "test:headed": "playwright test --headed",  
  "test:ui": "playwright test --ui",  
  "test:debug": "playwright test --debug",  
  "report": "playwright show-report",  
  "codegen": "playwright codegen",  
  "format": "prettier --write ."  
}
```

কেন: লম্বা কমান্ড বারবার টাইপ করার দরকার পড়বে না। **UI mode** (`npm run test:ui`) নতুনদের জন্য সবচেয়ে কাজের, এখানে প্রতিটা স্টেপ চোখে দেখা যায়।

7

Prettier

root এ `.prettierrc` :

```
{ "singleQuote": true, "semi": true, "printWidth": 100 }
```

কেন: কোডের স্টাইল সবসময় একরকম থাকবে। commit এর আগে `npm run format` চালাবেন।

Login test

8 Codegen দিয়ে আসল locator খুঁজে বের করা

```
npx playwright codegen https://www.skreno.ai/login
```

Skreno-র login পেজ আসলে এরকম (Playwright এর snapshot থেকে নেওয়া):

- heading "Welcome back"
- textbox "Email" (placeholder: you@example.com)
- textbox "Password" (placeholder: Enter your password)
- button "Show password"
- button "Sign in"

শিক্ষা: প্রথমবার `input[name="email"]` লেখা হয়েছিল অনুমান করে। কিন্তু Skreno-র ইনপুটে এই attribute নেই, তাই ৩০ সেকেন্ড পরে timeout হয়েছিল। locator **কখনো অনুমান করে লিখবেন না**। codegen অথবা error-context.md এর snapshot দেখে লিখবেন।

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByRole('textbox', { name: 'Email' });
    this.passwordInput = page.getByRole('textbox', { name: 'Password' });
    this.loginButton = page.getByRole('button', { name: 'Sign in' });
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

কেন `getByRole('textbox', ...)`: পেজে "Show password" নামে একটা বাটনও আছে।

`getByLabel('Password')` আংশিক মিল ধরে বলে ইনপুট আর বাটন দুটোকেই পেতে পারে, তখন **strict mode violation** এরর আসবে। `getByRole('textbox')` শুধু ইনপুট বক্স খোঁজে।

শিক্ষা: VS Code এর "Quick Fix → Add missing method" দিয়ে `goto()` বানালে ভেতরে থাকে `throw new Error('Method not implemented.')`। এটা রেখে দিলে test শুরু হতেই fail করে। অটো-জেনারেটেড কোড সবসময় পড়ে দেখে তারপর রাখবেন।

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../../pages/LoginPage';

test.describe('Login', () => {
  test.use({ storageState: { cookies: [], origins: [] } });

  test('valid user can log in', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login(process.env.TEST_EMAIL!, process.env.TEST_PASSWORD!);
    await expect(page).toHaveURL(/dashboard/);
  });

  test('wrong password shows error', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login(process.env.TEST_EMAIL!, 'wrong-pass');
    await expect(page.getByText(/invalid|incorrect/i)).toBeVisible();
  });
});
```

কেন:

- প্রতিটা ফিচারের জন্য একটা positive আর অন্তত একটা negative test লিখবেন।
- `test.use({ storageState: { cookies: [], origins: [] } })` লাইনটা পর্ব ৪ এর পরে লাগে। এটা login test কে **logged-out** অবস্থায় চালায়। এটা না দিলে সেভ করা session নিয়ে /login এ গেলে সাইট সরাসরি dashboard এ পাঠিয়ে দেবে, আর Email ফিল্ড পাওয়া যাবে না।

headed mode এ test চলার সময় ব্রাউজারে হাত দেবেন না। আপনার ক্লিক বা টাইপ test এর ফলাফল বদলে দেয়।

CI: GitHub Actions

11 GitHub Secrets যোগ করা

1. repo খুলে যান **Settings** → **Secrets and variables** → **Actions** এ।
2. **Repository secrets** এ `BASE_URL` , `TEST_EMAIL` , `TEST_PASSWORD` যোগ করুন। Environment secrets এ রাখলে চলবে না।
3. নাম হুবহু মিলতে হবে, বড়-ছোট হাতের অক্ষরসহ।

সবচেয়ে বড় ফাঁদ: value তে শুধু কাঁচা লেখা দেবেন: `me@example.com` , এরকম না: `'me@example.com'` ।
dotenv local এ কোটেশন বাদ দেয়, কিন্তু GitHub Secrets দেয় না। আমাদের ক্ষেত্রে ঠিক এই কারণেই local এ test সবুজ ছিল কিন্তু CI তে লাল হচ্ছিল। secret এর value পরে আর দেখা যায় না, তাই এই ভুল ধরা কঠিন।

```

name: Playwright Tests
on:
  push:
    branches: [main, master]
  pull_request:
    branches: [main, master]
jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: lts/*
      - name: Install dependencies
        run: npm ci
      - name: Install Playwright Browsers
        run: npx playwright install --with-deps chromium
      - name: Run Playwright tests
        run: npx playwright test
      - env:
          BASE_URL: ${ secrets.BASE_URL }
          TEST_EMAIL: ${ secrets.TEST_EMAIL }
          TEST_PASSWORD: ${ secrets.TEST_PASSWORD }
      - uses: actions/upload-artifact@v4
        if: ${ !cancelled() }
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 30

```

কেন:

- "Run Playwright tests" স্টেপ **একটাই**, আর সেটা ব্রাউজার ইনস্টলের **পরে**, env সহ।
- `install --with-deps chromium`: শুধু chromium চালাচ্ছেন, তাই বাকি ব্রাউজার ডাউনলোড করে সময় নষ্ট করার দরকার নেই।
- CI তে `.env` থাকে না, তাই value আসে secrets থেকে।
- "Node.js 20 deprecated" আর "Ubuntu 26" শুধু সতর্কবার্তা। এগুলো fail এর কারণ না।

শিক্ষা: workflow ফাইল GitHub ওয়েবসাইটে এডিট করবেন না। সব পরিবর্তন local এ করে push করবেন। দুই জায়গায় এডিট করলে push rejected (fetch first) আসবে, তখন `git pull --rebase origin main` করে আবার push করতে হবে।

Login session আর Dashboard test

13 ধারণা: storageState

setup project → একবার login → playwright/.auth/user.json এ cookie সেভ

↓

chromium project → প্রতিটা test ওই cookie নিয়ে logged-in অবস্থায় শুরু হয়

কেন: dashboard এর ১০০টা test এ প্রতিবার login করলে সময় নষ্ট হয়, আর Skreno-র rate limit ও লেগে যায়। login হবে একবার, তারপর session সবাই ব্যবহার করবে।

14 tests/auth.setup.ts (rate limit সুরক্ষাসহ)

```
import { test as setup, expect } from '@playwright/test';
import fs from 'fs';
import { LoginPage } from '../pages/LoginPage';

const authFile = 'playwright/.auth/user.json';
const ONE_HOUR = 60 * 60 * 1000;

setup('authenticate', async ({ page }) => {
  const isFresh =
    fs.existsSync(authFile) && Date.now() - fs.statSync(authFile).mtimeMs < ONE_HOUR;
  setup.skip(isFresh, 'Saved session is still fresh, skipping login');

  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login(process.env.TEST_EMAIL!, process.env.TEST_PASSWORD!);
  await expect(page).toHaveURL(/dashboard/);
  await page.context().storageState({ path: authFile });
});
```

কেন:

- toHaveURL(/dashboard/) : login সফল হয়েছে কিনা নিশ্চিত করে তারপর session সেভ করে। না হলে একটা ফাঁকা session সেভ হয়ে যেত।
- setup.skip(isFresh, ...) : session ফাইল ১ ঘণ্টার কম পুরনো হলে আবার login করে না। বারবার login করলে Skreno দেখায় "Too many sign-in attempts", এটা সেটা ঠেকায়।
- CI তে প্রতিবার নতুন মেশিন, ফাইল থাকে না, তাই সেখানে login হয় একবারই।
- ফাইলের নাম .setup.ts দিয়ে শেষ, তাই সাধারণ test হিসেবে এটা আলাদা করে চলে না।

session নষ্ট হয়ে গেলে (logout বা পাসওয়ার্ড বদল): `rm playwright/.auth/user.json`

15 config এর projects

```
projects: [
  { name: "setup", testMatch: /\.*\setup\.ts/ },
  {
    name: "chromium",
    use: {
      ...devices["Desktop Chrome"],
      storageState: "playwright/.auth/user.json",
    },
    dependencies: ["setup"],
  },
],
```

কেন: dependencies: ["setup"] মানে chromium এর test শুরুর আগে setup চলতেই হবে। auth.setup.ts ফাইল না থাকলে ENOENT: user.json এরর আসে। তাই এই ব্লক বসানোর **আগে** setup ফাইল বানাতে হবে।

16 pages/DashboardPages.ts

```
import { Page, Locator } from '@playwright/test';

export class DashboardPage {
  readonly page: Page;
  readonly sidebar: Locator;
  readonly userMenu: Locator;

  constructor(page: Page) {
    this.page = page;
    this.sidebar = page.getByRole('navigation');
    this.userMenu = page.getByRole('banner').getByRole('button', { name: 'Jashim' });
  }

  async goto() {
    await this.page.goto('/dashboard');
  }

  sidebarLink(name: string): Locator {
    return this.sidebar.getByRole('link', { name, exact: true });
  }
}
```

কেন:

- **Locator chaining** (this.sidebar.getByRole('link', ...)): শুধু sidebar এর ভেতরে খোঁজে। header এও "Skreno" নামে একটা লিংক আছে, তাই খোঁজার জায়গা সীমিত রাখলে ভুল লিংকে ক্লিক হয় না।
- exact: true : নামের হুবহু মিল চায়।
- sidebarLink(name) : ১০টা লিংকের জন্য ১০টা আলাদা property না বানিয়ে একটা method।
- userMenu এ "Jashim" নাম hardcode করা আছে। অন্য account এ চালালে পরে বদলাতে হবে।

```
import { test, expect } from '@playwright/test';
import { DashboardPage } from '../../pages/DashboardPages';

const menuItems = [
  { name: 'Requests', url: /\dashboard\/requests/ },
  { name: 'Testimonials', url: /\dashboard\/testimonials/ },
  { name: 'Meetings', url: /\dashboard\/meetings/ },
  { name: 'Guides', url: /\dashboard\/guides/ },
  { name: 'Team', url: /\dashboard\/team/ },
  { name: 'Billing', url: /\dashboard\/billing/ },
  { name: 'Integrations', url: /\dashboard\/integrations/ },
  { name: 'Settings', url: /\dashboard\/settings/ },
];

test.describe('Dashboard sidebar', () => {
  test('dashboard opens when already logged in', async ({ page }) => {
    const dashboard = new DashboardPage(page);
    await dashboard.goto();
    await expect(page).toHaveURL(/dashboard/);
    await expect(dashboard.sidebar).toBeVisible();
  });

  for (const item of menuItems) {
    test(`sidebar "${item.name}" opens correct page`, async ({ page }) => {
      const dashboard = new DashboardPage(page);
      await dashboard.goto();
      await dashboard.sidebarLink(item.name).click();
      await expect(page).toHaveURL(item.url);
    });
  }
});
```

কেন:

- প্রথম test এ login এর কোনো কোড নেই, তবুও dashboard খোলে। storageState কাজ করছে কিনা এটাই তার প্রমাণ।
- **Data-driven test:** for loop দিয়ে ৮টা test তৈরি হয়। নতুন মেনু এলে শুধু menuItems এ এক লাইন যোগ করলেই হবে।
- Community (বাইরের সাইট) আর "Earn 20%" বাদ রাখা হয়েছে।
- URL এ regex: Billing আসলে যায় /dashboard/billing/overview এ, শেষের অংশ বদলালেও test ভাঙবে না।

নির্দিষ্ট test চালানোর উপায়

কী চালাবেন	কমান্ড
সব test	<code>npx playwright test</code>
একটা ফোল্ডার	<code>npx playwright test tests/dashboard --headed</code>
ফাইলের নামের অংশ দিয়ে	<code>npx playwright test navigation</code>
নির্দিষ্ট লাইনের test	<code>npx playwright test tests/dashboard/navigation.spec.ts:24</code>
test এর নাম দিয়ে (grep)	<code>npx playwright test -g "Billing"</code>
নির্দিষ্ট project	<code>npx playwright test --project=chromium</code>
সব test এর তালিকা	<code>npx playwright test --list</code>
npm script দিয়ে, সাথে ফোল্ডার	<code>npm run test:headed -- tests/dashboard</code>

মনে রাখুন:

- **project = ফোল্ডার**। `--project` দিয়ে config এর project বাছাই হয় (`setup` , `chromium`)। `--project=dashboard` লিখলে এরর আসবে।
- `--` লাগে শুধু `npm run` এর সাথে। `npx playwright test -- --headed` লিখলে `--headed` কে ফাইলের নাম ধরে নেবে।
- `chromium` এর আগে `setup` সবসময় চলে। output এ `[setup]` দেখা স্বাভাবিক, আর session তাজা থাকলে পাশে `skipped` দেখাবে।
- `injected env (0) from .env` দেখে ঘাবড়াবেন না। এটা worker এর লাইন, main process আগেই ৩টা variable লোড করে ফেলেছে।

যে এরগুলো এসেছিল আর তার সমাধান

এরর	কারণ	সমাধান
Error: Method not implemented.	VS Code quick-fix এর বানানো ফাঁকা goto()	async goto() { await this.page.goto('/login'); }
ENOENT ... playwright/.auth/user.json	setup ফাইল ছাড়াই storageState চালু করা হয়েছিল, বা ভুল project কमेंট করা হয়েছিল	আগে auth.setup.ts বানান, তারপর storageState
প্রতিটা test দুইবার চলছিল	config এ একই নামে দুটো chromium project	একটা রাখুন
locator.fill: Test timeout ... input[name="email"]	অনুমান করে লেখা CSS locator পেজে মেলে না	getByRole('textbox', { name: 'Email' })
! [rejected] main -> main (fetch first)	GitHub ওয়েবে workflow এডিট করা হয়েছিল	git pull --rebase origin main, তারপর git push
CI তে test ব্রাউজার ছাড়া চলছিল	ওয়েব এডিটে test স্টেপ browser install এর আগে বসেছিল	একটাই test স্টেপ, install এর পরে
local সবুজ, CI লাল	Secret এর value তে ' কোটেশন	Secret আপডেট করুন, কোটেশন ছাড়া
Project(s) "dashboard" not found	ফোল্ডারের নাম --project এ দেওয়া হয়েছিল	npx playwright test tests/dashboard
Too many sign-in attempts	প্রতিবার setup নতুন করে login করছিল, সাথে wrong-password test এর ভুল login	১-২ মিনিট অপেক্ষা করুন। setup এ session freshness check রাখুন

ডিবাগ করার নিয়ম: test fail করলে আগে test-results/.../error-context.md খুলে **Page snapshot** অংশটা পড়বেন। সেখানে লেখা থাকে fail হওয়ার মুহুর্তে পেজে কী ছিল। তারপর trace দেখবেন: npx playwright show-trace। CI তে fail করলে Actions থেকে **playwright-report** artifact ডাউনলোড করবেন।

মেন্টরের পরামর্শ আর পরের ধাপ

1. **push** এর আগে **local** এ পাস করিয়ে নিন। **local** এ fail করলে CI তেও fail করবেই।
2. **page.waitForTimeout()** কখনো ব্যবহার করবেন না। Playwright নিজেই auto-wait করে। অপেক্ষা করাতে চাইলে **expect(...).toBeVisible()** ।
3. প্রতিটা **test** স্বাধীন রাখুন। একটা test আরেকটার ফলাফলের ওপর নির্ভর করবে না।
4. এক ধাপ পুরো শেষ করে তারপর পরেরটায় যাবেন। আগেভাগে কোড বসালে ENOENT এর মতো এরর আসে।
5. **wrong-password test** বারবার চালাবেন না। dashboard এ কাজ করার সময় শুধু **tests/dashboard** চালাবেন।
6. আলাদা **test account** নিন। Skreno ডেভ টিমকে বলুন staging এ test account কে rate limit থেকে ছাড় দিতে।
7. **Chrome extension test** পরে। এর জন্য **launchPersistentContext** লাগে।

পরের সম্ভাব্য কাজ

- Logout test (**userMenu** ব্যবহার করে)
- Custom fixture বানানো, যাতে প্রতিটা test এ **new DashboardPage(page)** লিখতে না হয়
- Recordings পেজের test: তালিকা, search, রেকর্ডিং খোলা
- Settings → Account এ ফর্ম test (faker দিয়ে)

কোনো ধাপে আটকে গেলে এররটা নিয়ে আসুন, তখন একসাথে ঠিক করব। 🚀